Phase 4: NLP & Text Analysis

Presentation Confidence Detector

Project Blueprint

Phase 4: NLP & Text Analysis

What Is This Phase?

This phase teaches you how to analyze **what someone says** (the transcript text) to detect confidence patterns. The Speech-to-Text engine from Phase 2 gives you raw text. Now you need to ANALYZE that text for signals like filler words, hedging, repetition, and sentence structure.

This is entirely NEW territory. ExamGuard never touched text analysis. Everything in this phase is a skill you have not used before.

Think of it this way:

- Phase 2 (Speech-to-Text) = the ears (turning sound into words)
- Phase 4 (NLP Text Analysis) = the language brain (understanding WHAT those words reveal about confidence)

WHY This Phase Matters for the Confidence Detector

A confident speaker and a nervous speaker say the SAME content differently:

```
Confident:  "Our revenue grew 40% this quarter."
Nervous:    "So, um, our revenue, like, grew... I think it was, um,
             around 40%? Yeah, so, this quarter, basically."
```

The camera cannot see this. The audio pitch analyzer cannot see this. Only TEXT analysis can catch:

- Filler words (um, uh, like, you know)
- Hedging phrases ("I think", "sort of", "maybe")
- Repetition (saying the same word three times in a row)
- Abandoned sentences (starting a thought and never finishing)
- Speaking pace (rushing = nervous, steady = confident)

Text analysis is one of the four pillars of your confidence score. Without it, you are missing the most obvious signals humans use to judge confidence.

Skills to Learn

#	Skill	What It Is	WHY for Confidence Detector	What to Learn	Difficulty	New?
1	Tokenization	Splitting text into individual words (tokens)	Every other text analysis skill depends on having clean, separated words to work with	`str.split()`, handling punctuation, lowercasing, what a "token" is	Easy	NEW
2	Pattern Matching	Checking text against a list of known words	The core technique behind filler	`in` operator, list lookups, `re` module	Easy	NEW

		or phrases	detection, hedging detection, and coaching alerts	basics, case-insensitive matching		
3	Filler Word Detection	Building a dictionary of filler words and counting them in speech	Filler words (um, uh, like, so, basically) are the #1 signal of nervousness — high filler rate = low confidence score	Build a filler dictionary, count occurrences, calculate filler ratio (fillers / total words), flag high-frequency fillers	Easy	NEW
4	Hedging Phrase Detection	Multi-word pattern matching for uncertainty phrases	Phrases like "I think", "sort of", "kind of", "I guess" signal the speaker does not believe what they are saying — direct hit to confidence score	Multi-word matching (not just single words), building a hedge phrase list, sliding window over tokens, partial matches	Medium	NEW
5	Repetition Detection	Finding consecutive duplicate words or phrases	Repeating words ("the the the") or phrases ("we need to we need to") signals the speaker lost their train of thought — a nervousness marker	Comparing adjacent tokens, sliding window for phrase repetition, stutter detection, distinguishing intentional repetition from nervous repetition	Medium	NEW
6	Speaking Pace Calculation	Computing words per minute from transcript + timestamps	Too fast (>180 WPM) = rushing/nervous, too slow (<100 WPM) = uncertain/lost, steady (120-150 WPM) = confident — pace is a strong	Words per minute formula, using STT timestamps, rolling average over 30-second windows, pace variance	Medium	NEW

			confidence signal	(consistency)		
7	Sentence Structure Analysis	Detecting complete vs. abandoned/fragment sentences	Confident speakers finish their sentences. Nervous speakers start a thought, abandon it, start over. Detecting this pattern adds depth to the score	End-of-sentence detection (periods, question marks), fragment detection (very short segments), restart detection ("I mean", "what I meant was")	Hard	NEW
8	Text Scoring	Combining multiple text metrics into a single speech confidence score	You have filler count, hedge count, repetition count, pace, and structure quality — you need ONE number (0-100) that represents speech confidence	Weighted scoring formula, normalizing different metrics to 0-1 range, defining thresholds (what filler rate = "bad"?), testing and tuning weights	Hard	NEW

Skill Details

1. Tokenization

What: Splitting a transcript string into individual words.

You tokenize by lowercasing the transcript and splitting it into individual words. A simple `split()` leaves punctuation attached to words, so a better approach uses a regex pattern to extract only word characters, producing clean tokens like "um", "i", "think" without trailing commas or periods.

Confidence Detector connection: Every analysis function takes a list of tokens as input. Tokenization is step 1 of the NLP pipeline.

---

2. Pattern Matching

What: Checking if words from the transcript appear in a known list.

You define a set of known filler words and then filter the token list to find any matches. For example, given the tokens from "Um, I think our revenue grew, like, 40 percent", the pattern matcher would find "um" and "like" as fillers.

Confidence Detector connection: Pattern matching is the engine behind filler detection, hedge detection, and restart detection. It is simple but it powers 60% of your text analysis.

---

3. Filler Word Detection

What: Counting filler words and calculating a filler ratio.

You define a comprehensive set of filler words (um, uh, er, ah, like, so, basically, literally, actually, honestly, right, okay, well, anyway, yeah) and a `detect_fillers` function that counts how many tokens match, calculates a filler ratio (fillers divided by total words), and returns both the count and the list of fillers found. The quality thresholds are: below 0.03 ratio is excellent, below 0.08 is good, below 0.15 needs work, and 0.15 or above is poor.

Confidence Detector connection: The filler ratio feeds directly into the text confidence score. It also triggers real-time coaching alerts like "Try to reduce filler words."

---

4. Hedging Phrase Detection

What: Finding multi-word phrases that signal uncertainty.

You define a list of multi-word hedge phrases ("i think", "i guess", "sort of", "kind of", "maybe", "perhaps", "probably", "not sure", etc.) and a `detect_hedges` function that searches the full lowercase transcript string for each phrase, counting occurrences. This works on the full string rather than individual tokens because hedges are multi-word patterns -- "I think" is hedging, but the word "think" alone in "think big" is not.

Why multi-word? Single-word detection misses hedges. "I think that is correct" is hedging. "Think big" is not. You need to match the full phrase "i think" to catch the hedge.

Confidence Detector connection: High hedging rate tells the system the speaker lacks conviction, even if their voice sounds steady.

---

5. Repetition Detection

What: Finding consecutive duplicate words or phrases.

You define a `detect_repetitions` function that scans the token list for two types of repetition. First, it checks for immediate stutters where the same word appears consecutively ("the the"). Second, it uses a sliding window to extract short phrases and checks if that same phrase appears later in the text, catching cases like "we need to we need to." Each repetition is recorded with its position and type (stutter vs phrase repeat).

Confidence Detector connection: Repetition reveals when the speaker's brain "stalls" — they repeat words while thinking of what to say next. This is a nervousness signal distinct from fillers.

6. Speaking Pace Calculation

What: Computing words per minute using transcript word count and timestamps from STT.

You define a `calculate_pace` function that divides word count by elapsed seconds and multiplies by 60 to get words per minute. A `pace_score` function converts WPM to a 0-100 score based on ideal ranges: 120-150 WPM scores 100 (perfect), 100-120 or 150-180 scores 75 (slightly off), 80-100 or 180-200 scores 50 (noticeably too slow or fast), and anything outside that scores 25. For meaningful results, pace should be calculated over a rolling 30-second window rather than the entire session, so you can see pace changes during the presentation.

Confidence Detector connection: Pace feeds into the text score AND triggers coaching alerts ("You are speaking too fast — try to slow down").

---

7. Sentence Structure Analysis

What: Detecting whether the speaker finishes their thoughts or abandons them mid-sentence.

You define an `analyze_structure` function that splits the transcript on sentence-ending punctuation and classifies each segment. Segments containing restart markers ("i mean", "let me rephrase", "sorry", "wait") are counted as restarts. Very short segments with fewer than 4 words are counted as fragments (abandoned thoughts). Everything else counts as a complete sentence. The function returns totals for each category, revealing how often the speaker finishes their thoughts versus abandoning and restarting them.

Confidence Detector connection: A speaker who completes 90% of their sentences scores high. A speaker who restarts constantly and leaves fragments everywhere scores low. This is the most nuanced text signal.

---

8. Text Scoring

What: Combining all text metrics into one speech confidence score (0-100).

You define a `calculate_text_score` function that normalizes each metric to a 0-1 scale (where 1 is confident) and combines them with weights: fillers at 30%, hedges at 20%, pace at 20%, repetition at 15%, and sentence structure at 15%. Each metric is normalized relative to its "bad" threshold (e.g., filler ratio approaching 0.15 drives the filler score toward 0). The weighted sum is multiplied by 100 to produce a final 0-100 text confidence score that feeds into the overall combined score in Phase 5. These weights and thresholds are MVP heuristics, not fixed scientific truth, so you should tune them after testing on real transcripts.

Confidence Detector connection: This is THE output of Phase 4. One number, 0-100, representing speech text confidence. This feeds into the final combined score in Phase 5 alongside face score and voice score. STT is the upstream text source, not a separate user-facing score.

Complete Hedging Phrase List

Every hedging phrase the Confidence Detector detects, with WHY each one signals low confidence:

Uncertainty Hedges

These phrases directly express doubt. The speaker is telling the audience "I am not sure about what I am saying."

Phrase	Example	WHY It Signals Low Confidence
"I think"	"I think our revenue grew"	The speaker is not sure. Compare: "Our revenue grew" — no doubt. Adding "I think" downgrades a fact to an opinion.
"I believe"	"I believe the deadline is Friday"	Same as "I think" but slightly more formal. Still expresses uncertainty rather than stating a fact.
"I feel like"	"I feel like we should change direction"	Frames a business decision as an emotion. Sounds unsure because feelings can be wrong; facts cannot.
"I guess"	"I guess the results are positive"	The weakest possible endorsement. "I guess" implies the speaker barely agrees with their own statement.

Probability Hedges

These words soften definitive statements by introducing probability. Instead of "this will work," the speaker says "this might work."

Phrase	Example	WHY It Signals Low Confidence
"maybe"	"Maybe we should launch in Q3"	A proposal wrapped in doubt. The audience hears "I am not committed to this."
"probably"	"This will probably increase revenue"	Sounds less confident than "This will increase revenue." The speaker is hedging against being wrong.
"perhaps"	"Perhaps we could consider..."	Double hedge — "perhaps" + "could." Very tentative. Common in academic speech but weak in presentations.

Softeners

These weaken the speaker's position by making statements less definitive. They are closely related to the filler words "sort of" and "kind of" from Phase 2.

Phrase	Example	WHY It Signals Low Confidence
"sort of"	"We sort of achieved our targets"	Did you achieve them or not? "Sort of" means "not really." The speaker is preemptively softening a weak result.
"kind of"	"It kind of works in most cases"	Same effect as "sort of." The audience hears: "It does not fully work."
"a little bit"	"We need to improve a little bit"	Minimizing the problem. Often used when the improvement needed is actually significant, but the speaker is afraid to say so.

Disclaimers

These phrases explicitly invite the audience to doubt or correct the speaker. They signal the speaker expects to be wrong.

Phrase	Example	WHY It Signals Low Confidence
"I'm not sure"	"I'm not sure, but I think it was 40%"	The speaker just told the audience to not trust the number that follows.
"I could be wrong"	"I could be wrong, but our costs went down"	Pre-emptive apology. The speaker is protecting themselves from challenge before making a claim.
"correct me if I'm wrong"	"Correct me if I'm wrong, but..."	Invites the audience to contradict you before you have even finished the sentence.
"if I'm not mistaken"	"If I'm not mistaken, we shipped 500 units"	Implies the speaker might be mistaken. Either you know the number or you do not.

Apology Hedges

These are not true apologies — they are verbal habits that signal the speaker feels they are imposing or that their content is not worth the audience's time.

Phrase	Example	WHY It Signals Low Confidence

"sorry"	"Sorry, but I have a different view"	Apologizing for having an opinion. Confident speakers state their view without apology.
"sorry but"	"Sorry but the data shows otherwise"	The data does not require an apology. This is a subordination habit — the speaker ranks themselves below the audience.

Total: 16 hedging phrases. Each is detected by substring matching on the lowercase transcript. Note: "sort of" and "kind of" also appear in the Phase 2 filler word list. They are counted in BOTH places because they serve both functions — they are verbal fillers AND they hedge the statement. The scoring in Phase 5 accounts for this overlap so the penalty is not doubled.

WPM Calculation Walkthrough

Words Per Minute (WPM) is the simplest and most informative single metric for speaking pace.

The Formula

```
WPM = (total_words / elapsed_seconds) x 60
```

That is it. Count the words in the transcript, divide by how many seconds have passed, multiply by 60 to convert to per-minute rate.

Worked Example

A speaker gives a 2-minute practice talk. The transcript contains 280 words.

```
WPM = (280 / 120) x 60
WPM = 2.333 x 60
WPM = 140 WPM
```

140 WPM falls in the optimal range. The speaker is pacing well.

Another Example: Nervous Speaker

Same 2-minute window, but the transcript contains 390 words.

```
WPM = (390 / 120) x 60
WPM = 3.25 x 60
WPM = 195 WPM
```

195 WPM is too fast. The speaker is rushing — a classic nervousness signal.

The Thresholds

WPM Range	Category	What It Means	Score Impact
< 100	Too slow	Speaker is hesitant, uncertain, or reading word-by-word. Audience loses engagement.	Penalty increases as WPM drops

100 - 130	Slightly slow	Acceptable for complex technical content. May feel deliberate rather than nervous.	Minor penalty
130 - 160	**Optimal**	Ideal presentation pace. Clear, confident, easy to follow. TED Talk average is ~150 WPM.	No penalty (full score)
160 - 180	Slightly fast	Common in energetic speakers. Acceptable for short bursts, concerning if sustained.	Minor penalty
> 180	Too fast	Rushing. Audience cannot keep up. Words blur together. Strong nervousness signal.	Penalty increases as WPM rises

Rolling Window vs Session Average

The session average WPM hides important patterns. A speaker who talks at 120 WPM for 3 minutes and then 190 WPM for 1 minute averages 137 WPM — which looks optimal. But that last minute was clearly a rush.

Use a **30-second rolling window**: calculate WPM over the most recent 30 seconds of speech. This reveals pace changes in near-real-time and lets you show the user when they sped up or slowed down. The post-session report can include: "Your pace was 135 WPM for the first 3 minutes (excellent), then jumped to 190 WPM in the final minute (too fast — you may have been rushing to finish)."

Speech Scoring Weights Explained

The text scoring engine combines five metrics into one score. But they are NOT weighted equally. Here is WHY:

Metric	Maximum Penalty	Weight	WHY This Weight
Filler penalty	Up to 40 points	Highest	Fillers are the most noticeable distraction for an audience. A speaker who says "um" every 10 seconds is visibly nervous to everyone in the room. Audiences notice fillers before they

			notice hedging, pace issues, or repetition. Research from Toastmasters and presentation coaching consistently ranks fillers as the number one audience distraction.
Hedge penalty	Up to 30 points	High	Hedging is softer than fillers — an audience might not consciously notice "I think" or "sort of," but it erodes their trust in the speaker over time. Hedging is also sometimes appropriate: "I believe this will work" is more honest than "This will work" when the speaker is genuinely uncertain. We penalize it less harshly than fillers because context matters.
Repetition penalty	Up to 15 points	Moderate	Repetition happens less frequently than fillers or hedging. A speaker might repeat a phrase 2-3 times in a 5-minute talk. It signals a momentary brain stall, not a systemic habit. The audience notices but does not dwell on it. Lower weight reflects lower frequency and lower impact.
Pace penalty	Up to 15 points	Moderate	Pace is a supporting signal, not a primary one. A speaker at 170 WPM who has zero fillers and no hedging sounds energetic, not nervous. A speaker at 170 WPM who also says "um" constantly sounds panicked. Pace

			amplifies other signals but rarely drives the impression on its own. Also, optimal pace varies by culture and context, so we penalize it lightly.

Total maximum penalty: 100 points. A speaker with extreme fillers (40), constant hedging (30), repetition (15), and wild pace (15) scores 0. A clean speaker with no fillers, no hedging, no repetition, and perfect pace scores 100.

These weights are the NLP sub-pipeline weights, separate from the overall system weights. The overall Confidence Detector combines three signal sources: Face (0.40), Speech/Text (0.35), Voice (0.25). The NLP text score is one component within the Speech/Text channel.

The "Like" Problem

"Like" is the most controversial word in the filler dictionary. It is a legitimate English word that is ALSO the most common filler word among English speakers under 40.

The Problem

Sentence	Is "Like" a Filler?	How Our Pattern Matcher Scores It
"It was, like, really important"	YES — pure filler	Counted as filler (correct)
"I like this approach"	NO — verb meaning "enjoy"	Counted as filler (WRONG)
"It looks like rain"	NO — preposition meaning "similar to"	Counted as filler (WRONG)
"Like I said earlier"	BORDERLINE — conversational filler or legitimate reference	Counted as filler (debatable)
"She was like, 'no way'"	YES — quotative filler, extremely casual	Counted as filler (correct)

Our simple pattern matcher counts ALL instances of "like" as fillers. It does not understand grammar, sentence structure, or word function. This means:

- In a sentence with "I like pizza, and it was, like, amazing," our system counts TWO fillers when only ONE is actually a filler.
- Estimated false positive rate: **~15%** of "like" occurrences in presentation transcripts are legitimate uses, not fillers.

WHY We Accept This

Catching 85% of "like" fillers instantly is better than catching 0%. The alternative — not counting "like" at all — would miss the single most common filler word. A speaker who says "like" 30 times in a 5-minute talk is almost certainly using it as a filler for at least 25 of those instances.

The math: if a speaker uses "like" 30 times and 4-5 are legitimate, our filler count is inflated by 4-5 words. On a transcript of 700 words, that changes the filler ratio from 3.57% (25/700) to 4.28% (30/700). Both fall in the same quality band. The impact on the final score is negligible.

The v2 Fix

In v2, a language model (like Claude API) can understand context. You send the transcript with each "like" highlighted and ask: "Is this 'like' a filler word or a legitimate use?" The model can distinguish "I like pizza" (verb) from "It was, like, amazing" (filler) with high accuracy. This eliminates the false positive problem entirely — but it requires an API call, adds latency, and costs money per request. Not appropriate for the MVP, but a clear upgrade path.

Edge Cases

Real speech is messy. Here are the edge cases the Confidence Detector will encounter and how we handle (or do not handle) them:

Technical Jargon

A speaker presenting on machine learning might say: "The convolutional neural network uses backpropagation through stochastic gradient descent." The STT engine may misrecognize technical terms, and the NLP pipeline may flag pauses around jargon as filler-adjacent.

Our approach: We do not try to handle jargon specially. If the STT recognizes the word correctly, it passes through cleanly. If the STT misrecognizes it, we get a wrong word in the transcript — but this does not typically get counted as a filler (since "convolution" is not in the filler list). The main risk is WPM miscounting if the STT drops technical words entirely. Acceptable for MVP.

Non-English Filler Sounds

Speakers whose first language is not English often use filler sounds from their native language:

- Arabic speakers: "ya3ni" (means "I mean")
- Hindi speakers: "matlab" (means "I mean") or "acha" (means "okay")
- Turkish speakers: "yani" (means "so/I mean")
- Japanese speakers: "eto" or "ano"

Our approach: The MVP detects only English fillers. Non-English filler sounds will either be misrecognized by the STT (and not matched to our filler list) or ignored entirely. This is a known gap. A v2 improvement could add language-specific filler dictionaries, but the STT itself would also need to handle code-switching (mixing languages).

Reading from a Script (Zero Fillers)

If a speaker reads their presentation word-for-word from a script, they will have zero fillers, zero hedging, perfect sentence structure, and steady pace. The text score would be 100/100 — but the presentation would be terrible. Robotic, no audience connection, no spontaneity.

Our approach: The MVP does not penalize "too perfect" speech. The face score (eye contact looking down at script = low) and voice score (monotone pitch = low) will catch this. The three-signal system (Face 0.40, Speech 0.35, Voice 0.25) compensates: a perfect text score with terrible face and voice

scores still produces a low overall score. In v2, you could add a "naturalness" metric that flags suspiciously perfect text, but it is not needed when face and voice do their jobs.

Very Short Sessions (Under 30 Seconds)

WPM calculation becomes unreliable with very few words. If someone speaks 15 words in 8 seconds, WPM = (15/8) x 60 = 112 WPM. But 15 words is not enough to be meaningful — one long word could shift the rate by 10 WPM.

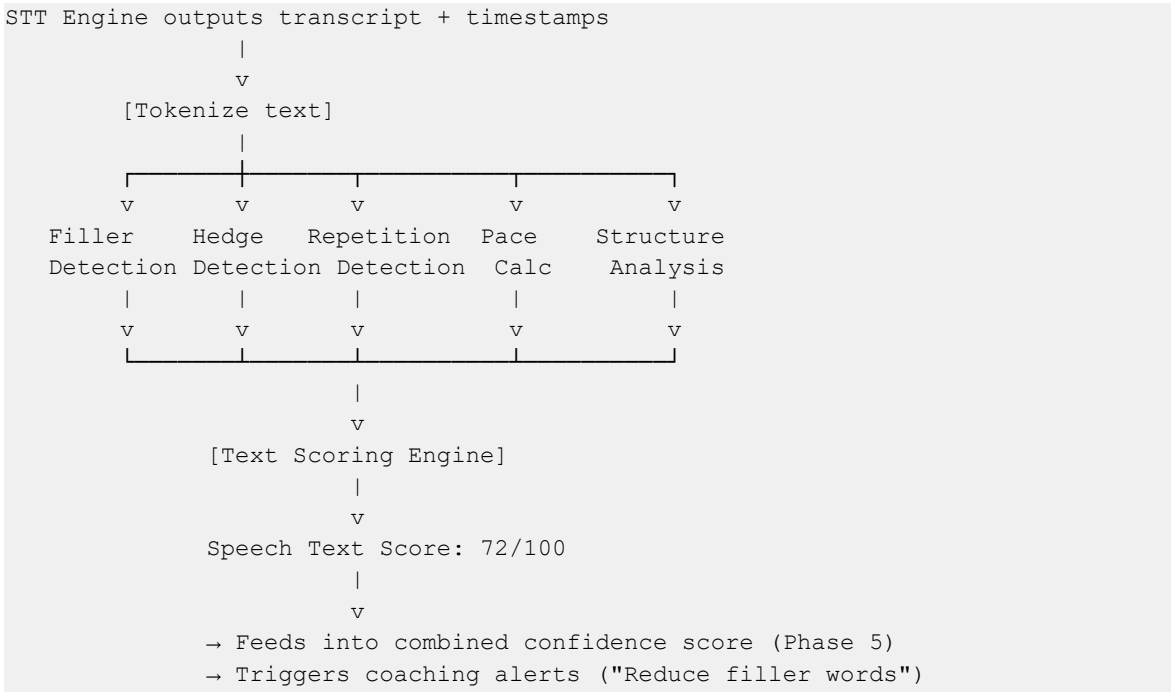
Our approach: Require a minimum of 30 seconds of speech before showing pace metrics. Display "Gathering data..." until enough speech has been captured. Filler and hedge detection can run immediately since they do not depend on duration.

Intentional Repetition vs Nervous Repetition

"We need change. Real change. Lasting change." — This is intentional rhetorical repetition (anaphora). "We need to, we need to, we need to change things" — This is a nervous stutter.

Our approach: The repetition detector counts both as repetitions in the MVP. Rhetorical repetition is relatively rare in practice talks (most people are not doing Martin Luther King Jr. speeches). The false positive rate is low enough to accept. In v2, sentence boundary detection could help — if the repeated word appears at the start of separate sentences, it is likely rhetorical; if it appears mid-sentence with restarts, it is likely nervous.

The NLP Pipeline — How It All Fits Together



Key Functions You Will Build

Function	Input	Output	Used For
`tokenize(transcript)`	Raw transcript	List of lowercase	Every other

	string	words	function
`detect_fillers(tokens)`	Token list	Filler count, ratio, list	Filler score + alerts
`detect_hedges(transcript)`	Full transcript string	Hedge phrases found + counts	Hedge score
`detect_repetitions(tokens)`	Token list	Repetition locations + types	Repetition score
`calculate_pace(word_count, seconds)`	Word count + duration	WPM number	Pace score + alerts
`analyze_structure(transcript)`	Full transcript string	Completion rate, fragments, restarts	Structure score
`calculate_text_score(...)`	All metrics above	Single score 0-100	Combined confidence score

---

Resources

Text Processing in Python

- [Python String Methods](https://docs.python.org/3/library/stdtypes.html#string-methods) — `split()`, `lower()`, `count()`, `replace()`
- [Python `re` Module](https://docs.python.org/3/library/re.html) — Regular expressions for pattern matching
- [Real Python: Regular Expressions](https://realpython.com/regex-python/) — Practical regex tutorial

NLP Concepts

- [NLTK Book Chapter 1](https://www.nltk.org/book/ch01.html) — Introduction to text processing (read for concepts, we use plain Python not NLTK)
- [spaCy 101](https://spacy.io/usage/spacy-101) — Modern NLP library overview (reference for later, not needed for MVP)

Filler Word Research

- [Harvard Business Review: How to Stop Saying Um](https://hbr.org/2018/08/how-to-stop-saying-um-ah-and-you-know) — Understanding why fillers matter
- [Toastmasters Filler Word Guide](https://www.toastmasters.org/) — Public speaking filler word awareness

Speaking Pace

- [Ideal Speaking Rate Research](https://www.ncbi.nlm.nih.gov/) — Search "speaking rate confidence" for academic sources
- General guideline: 120-150 WPM for presentations, 150-170 WPM for conversations

Similar Projects

- [Speech Analyzer GitHub repos](https://github.com/topics/speech-analysis) — See how others approach text-based speech analysis

- [Grammarly Tone Detector](https://www.grammarly.com/tone-detector) — Commercial product doing similar text analysis (for writing, not speech)